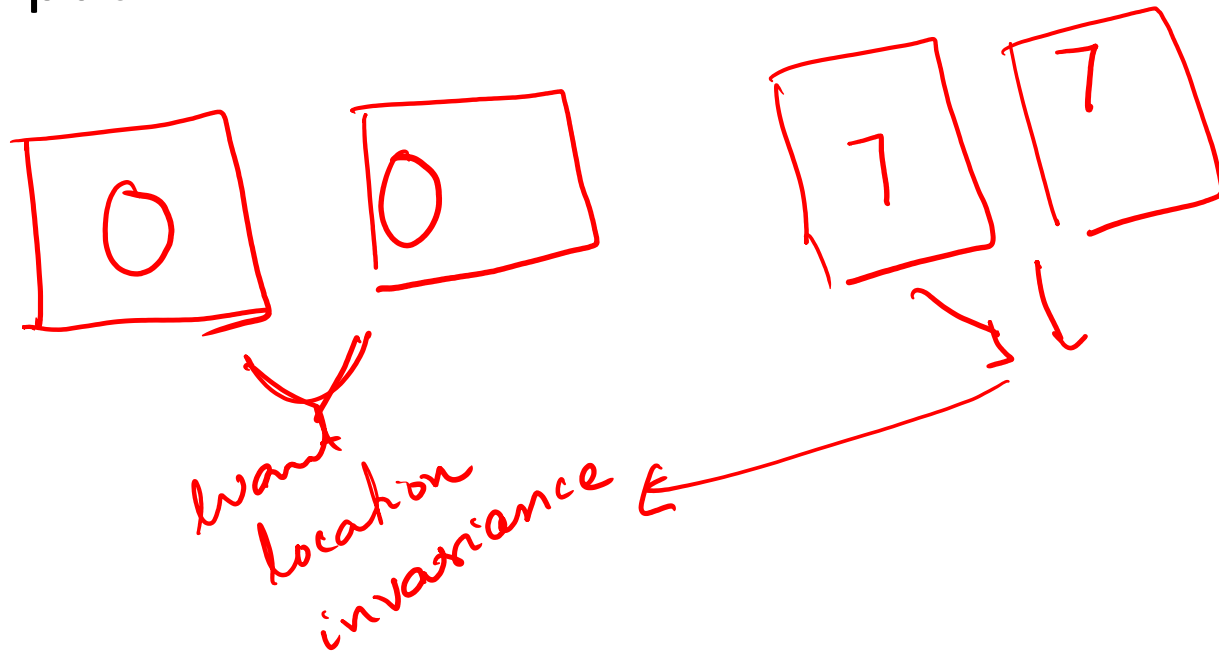
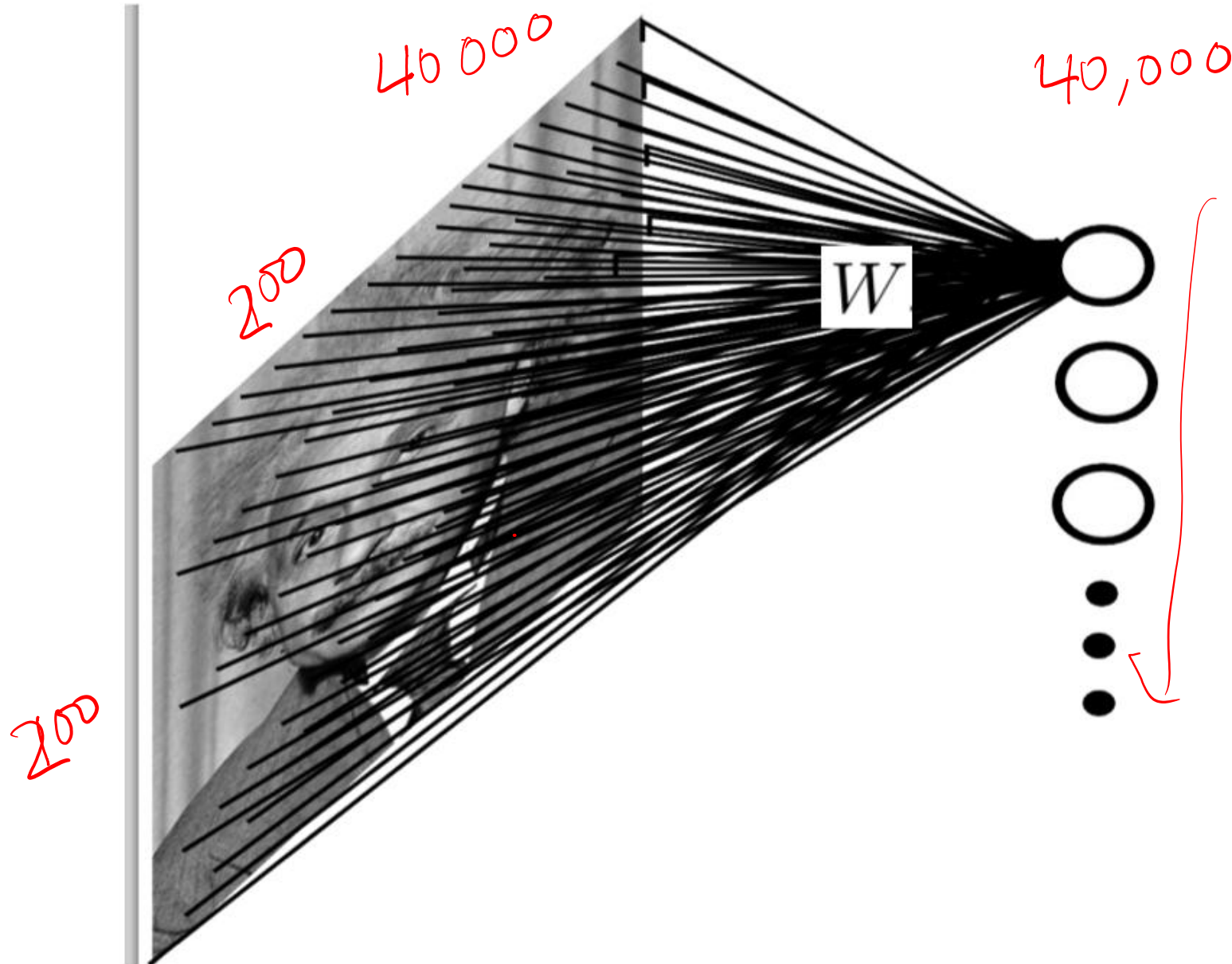


Convolution Neural Networks (CNNs)

- A special layer suitable when input features are regular and interchangeable
- Popularly used in images where object location is not pinned, variable length sequences, time series data
- Reasons for CNN: sparse connections, parameter sharing, variable length input

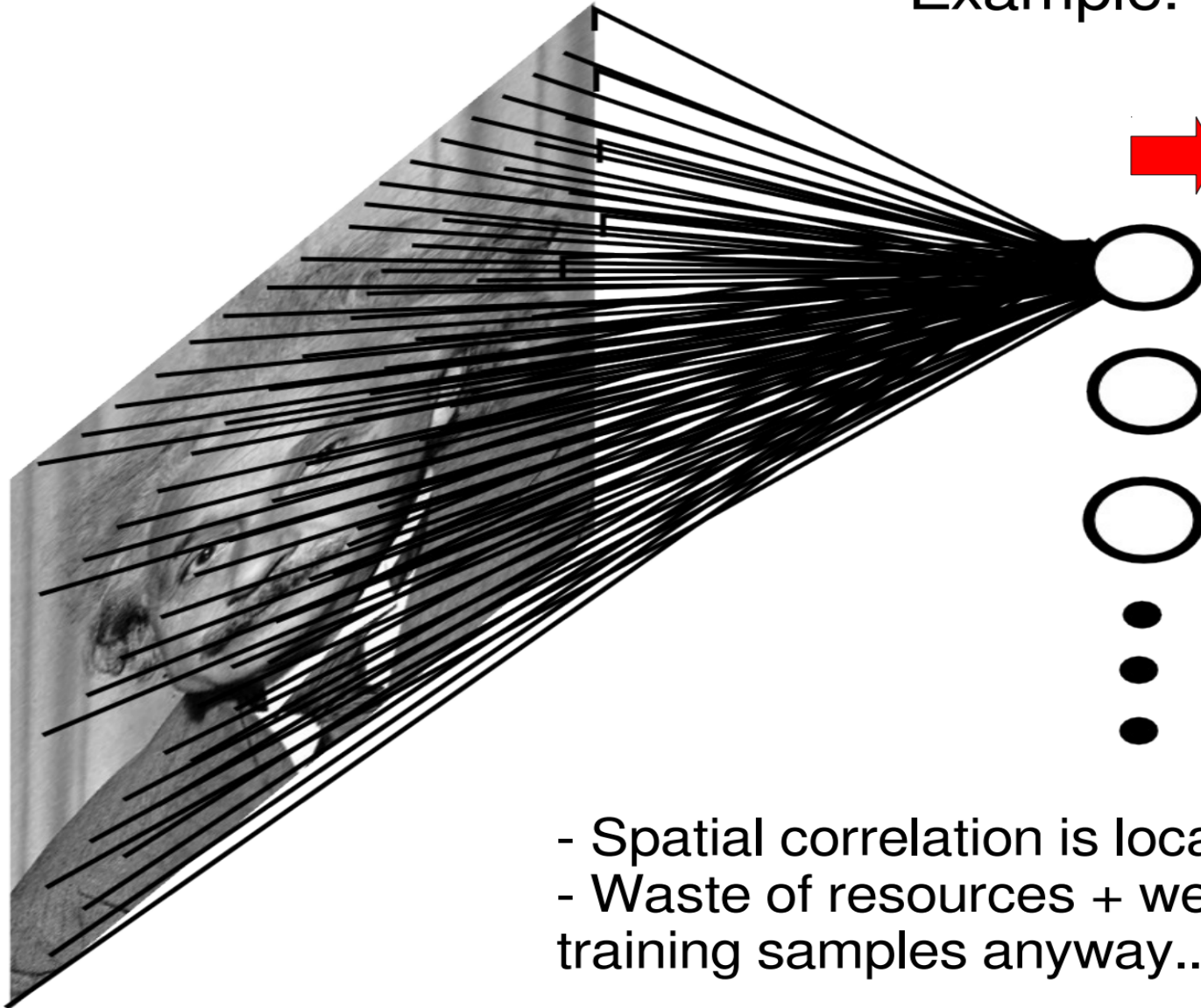


When the input data is an image..



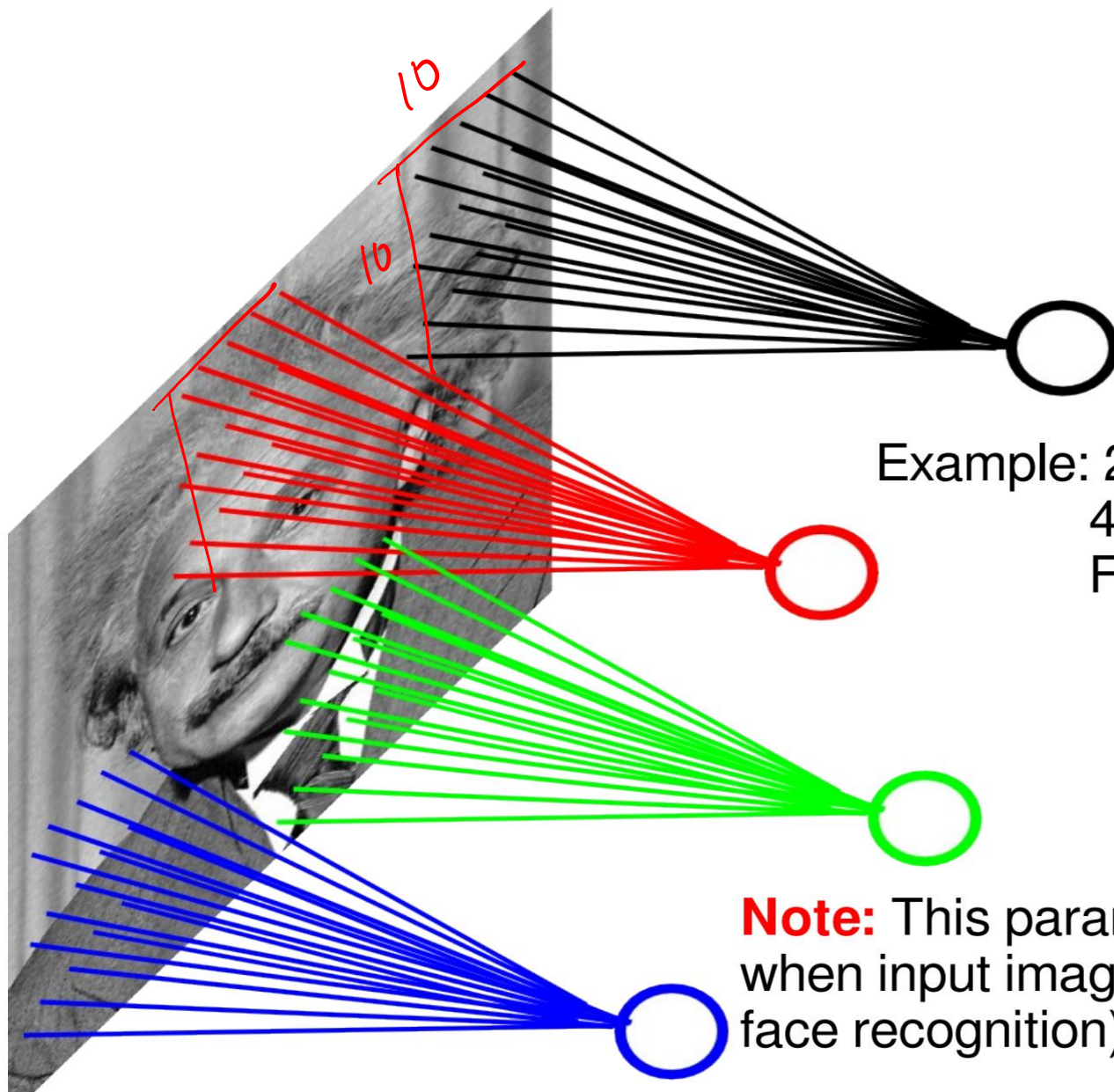
Fully Connected Layer

Example: 200x200 image
40K hidden units
~2B parameters!!!



- Spatial correlation is local
- Waste of resources + we have not enough training samples anyway..

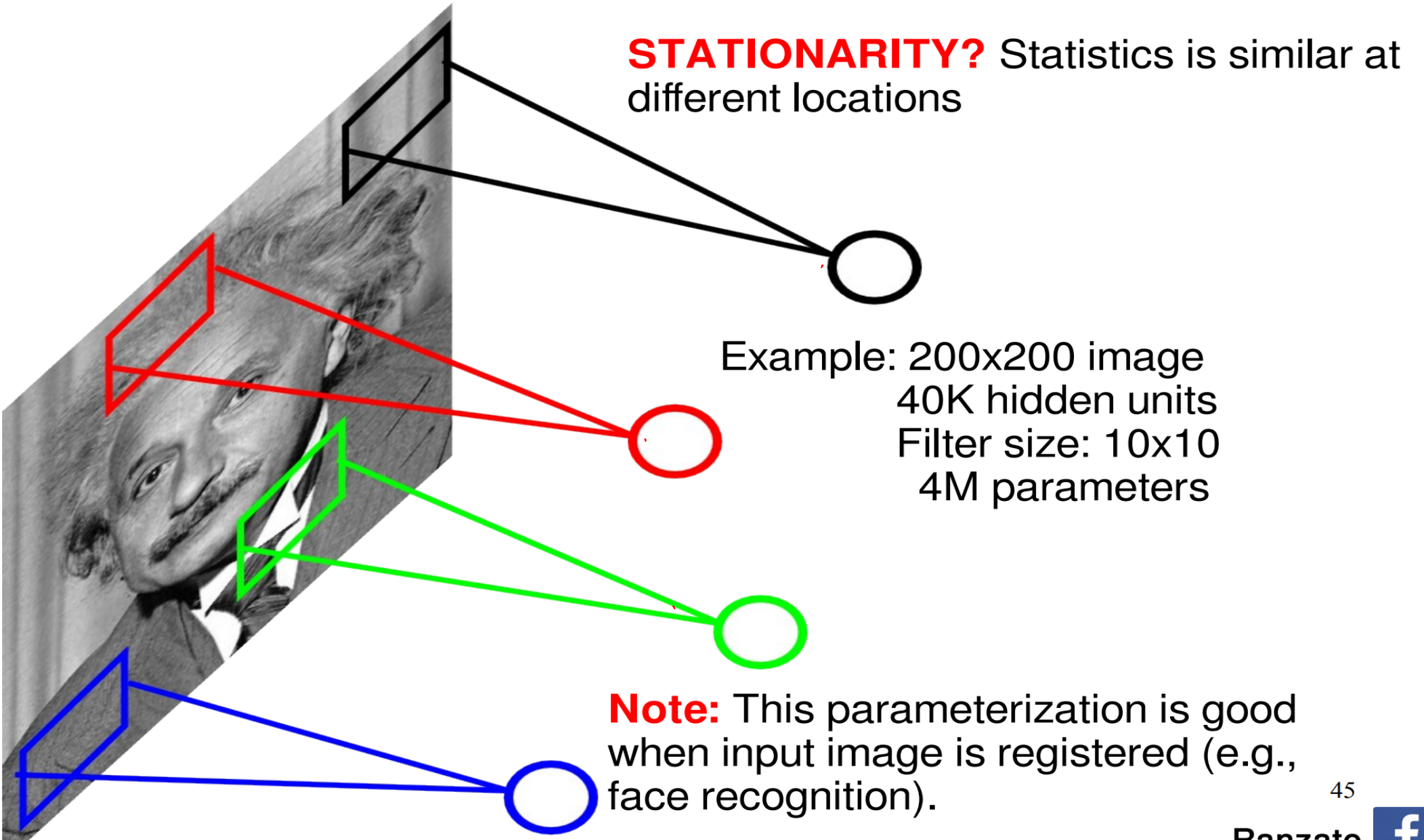
Locally Connected Layer



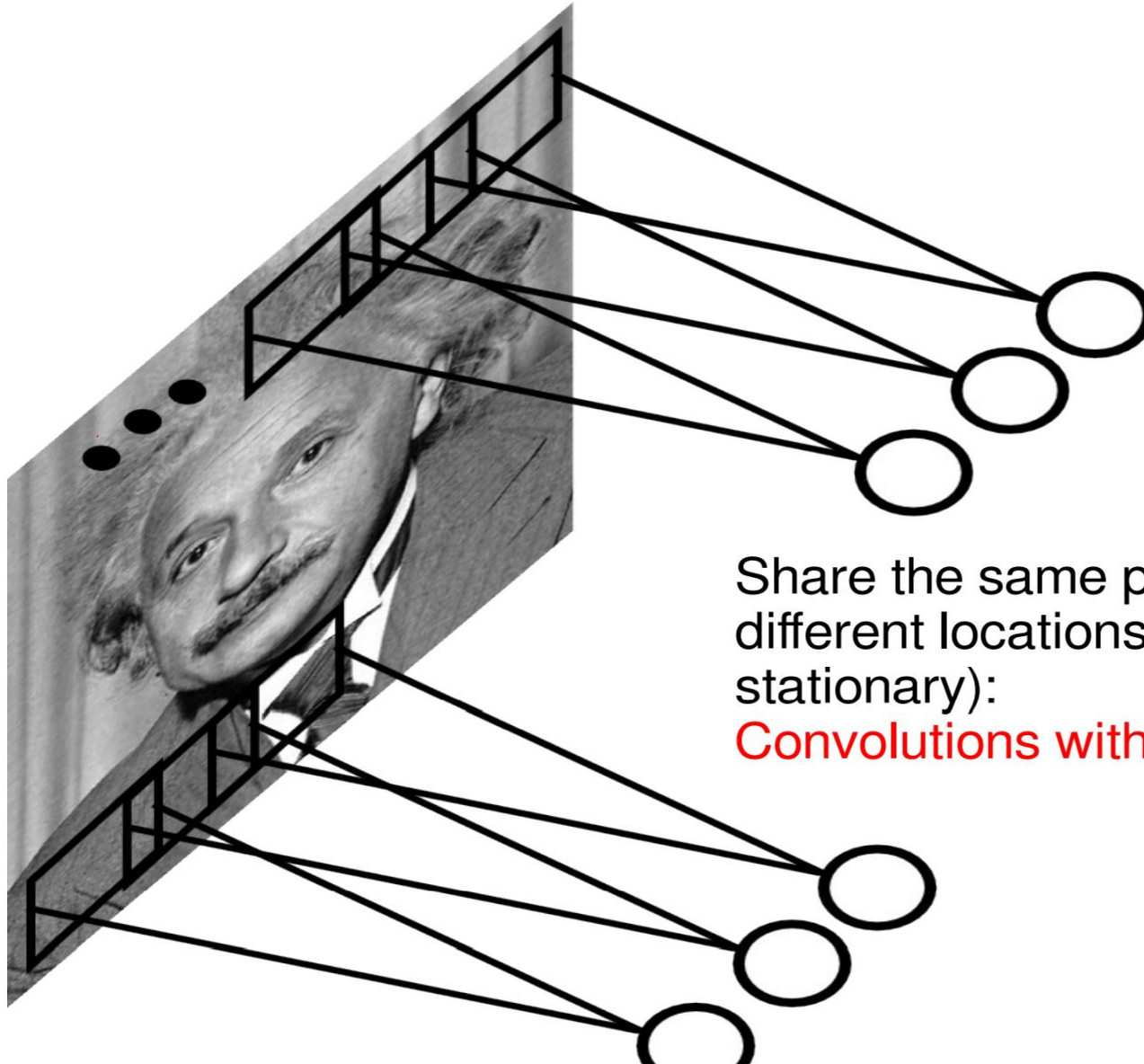
Example: 200x200 image
40K hidden units
Filter size: 10x10
4M parameters

Note: This parameterization is good when input image is registered (e.g., face recognition).

Locally Connected Layer



Convolutional Layer



Share the same parameters across different locations (assuming input is stationary):

Convolutions with learned kernels

Input

0	1	2
3	4	5
6	7	8

Kernel

 W^1

w_{11}	0	1	w_{12}
w_{21}	2	3	w_{22}

*

=

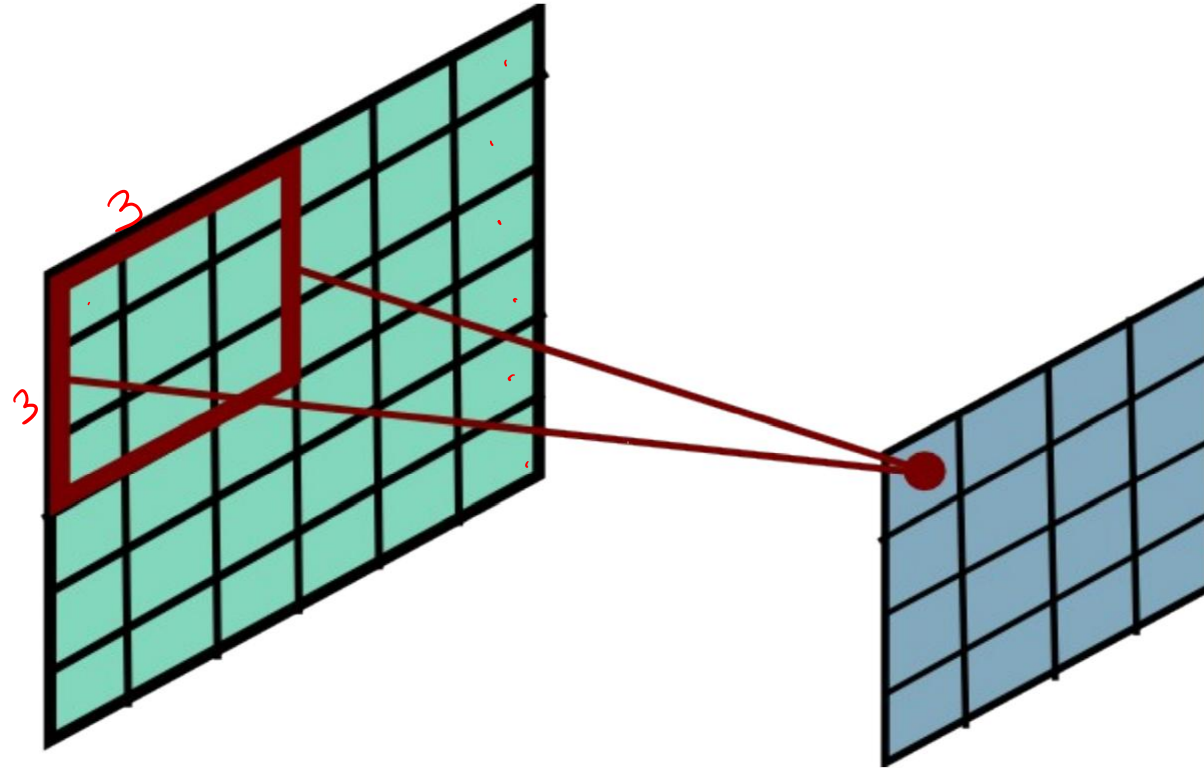
Output

19	25
37	43

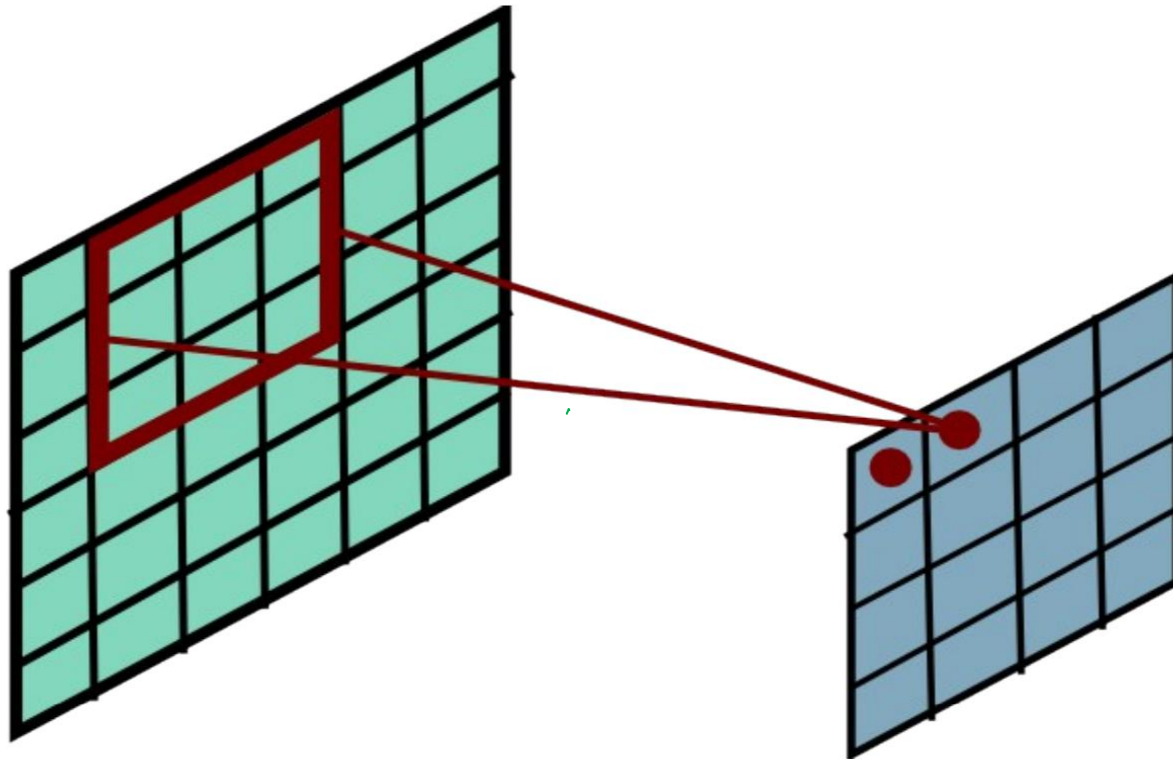
$$\rightarrow 0 \times 0 + 1 \times 1 + 2 \times 3 + 3 \times 4$$

$$\rightarrow 4 \times 0 + 1 \times 5 + 7 \times 2 + 3 \times 8$$

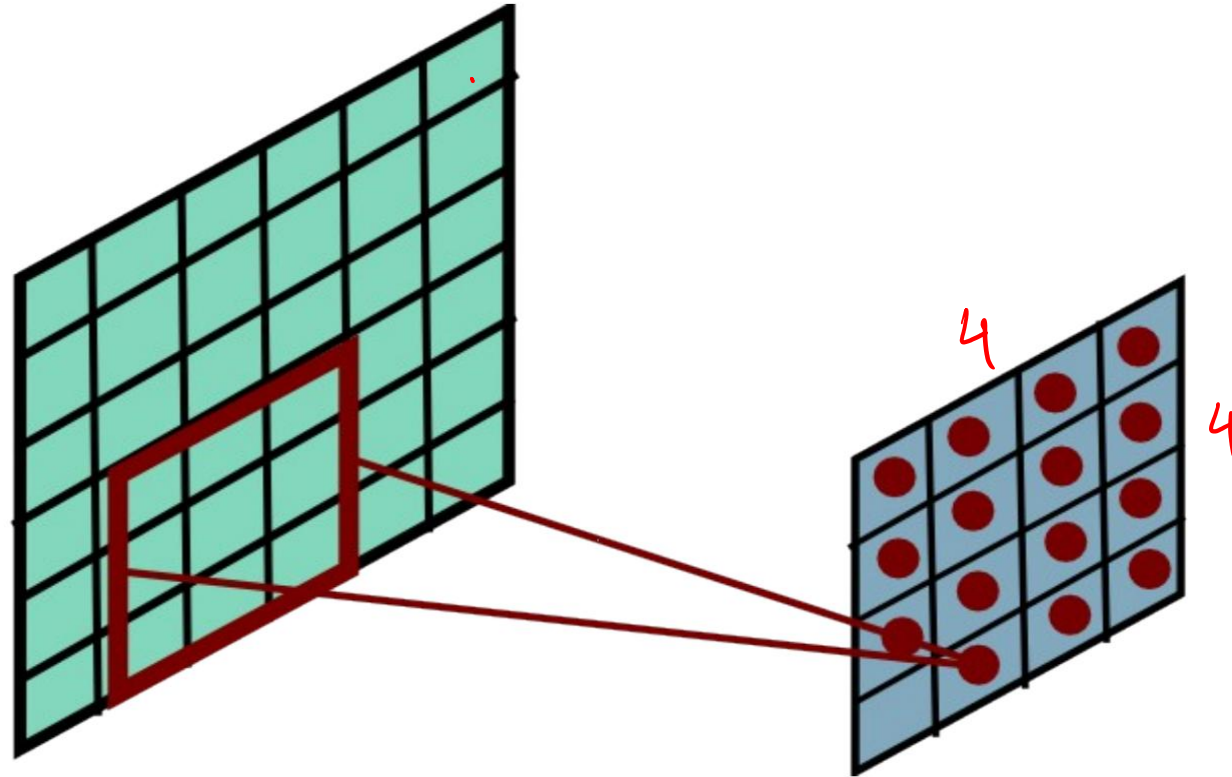
Convolutional Layer



Convolutional Layer



Convolutional Layer

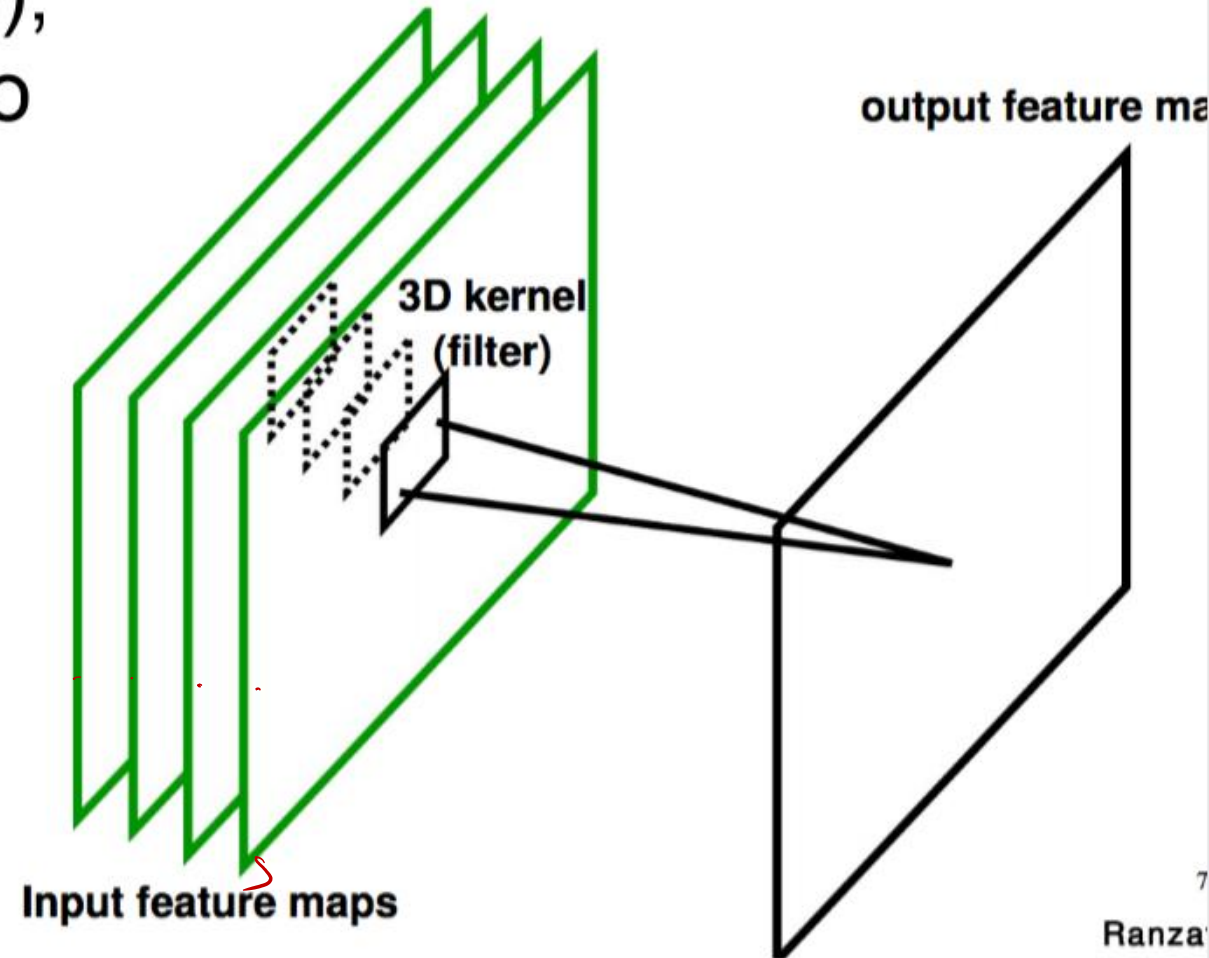


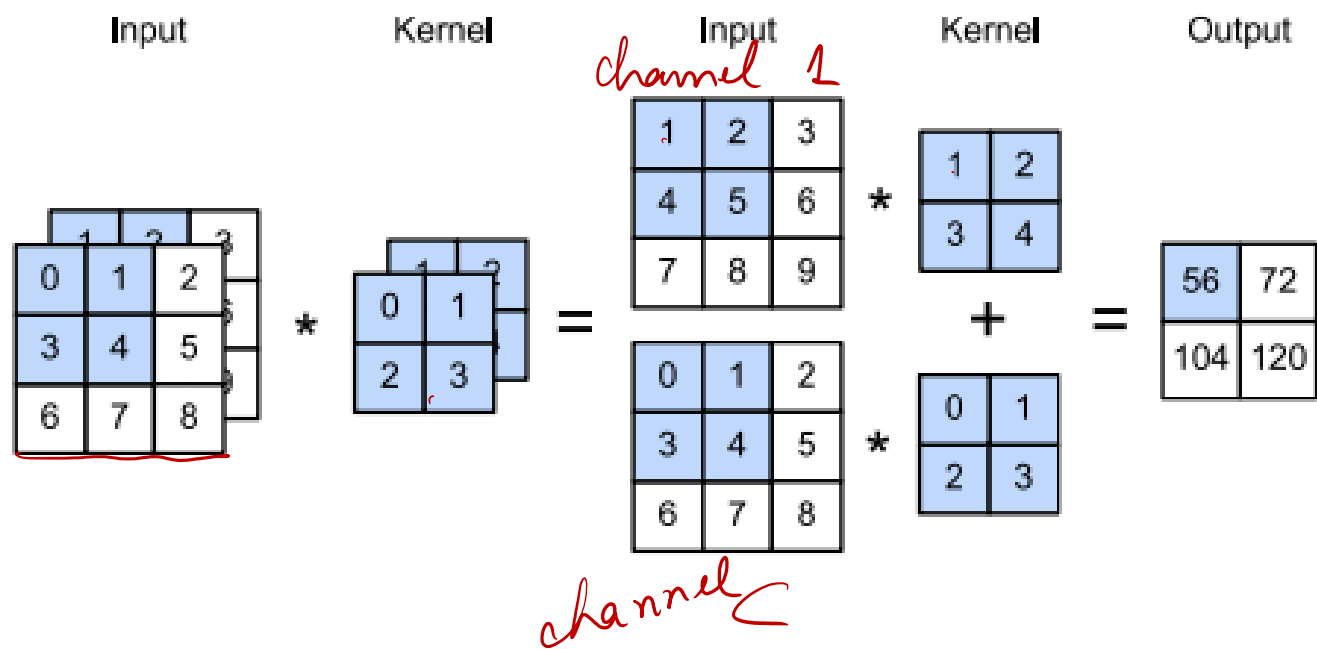
Detail

If the input has 3 channels (R,G,B), 3 separate k by k filter is applied to each channel.

Output of convolving 1 feature is called a *feature map*.

This is just sliding window, ex. the output of one part filter of DPM is a feature map

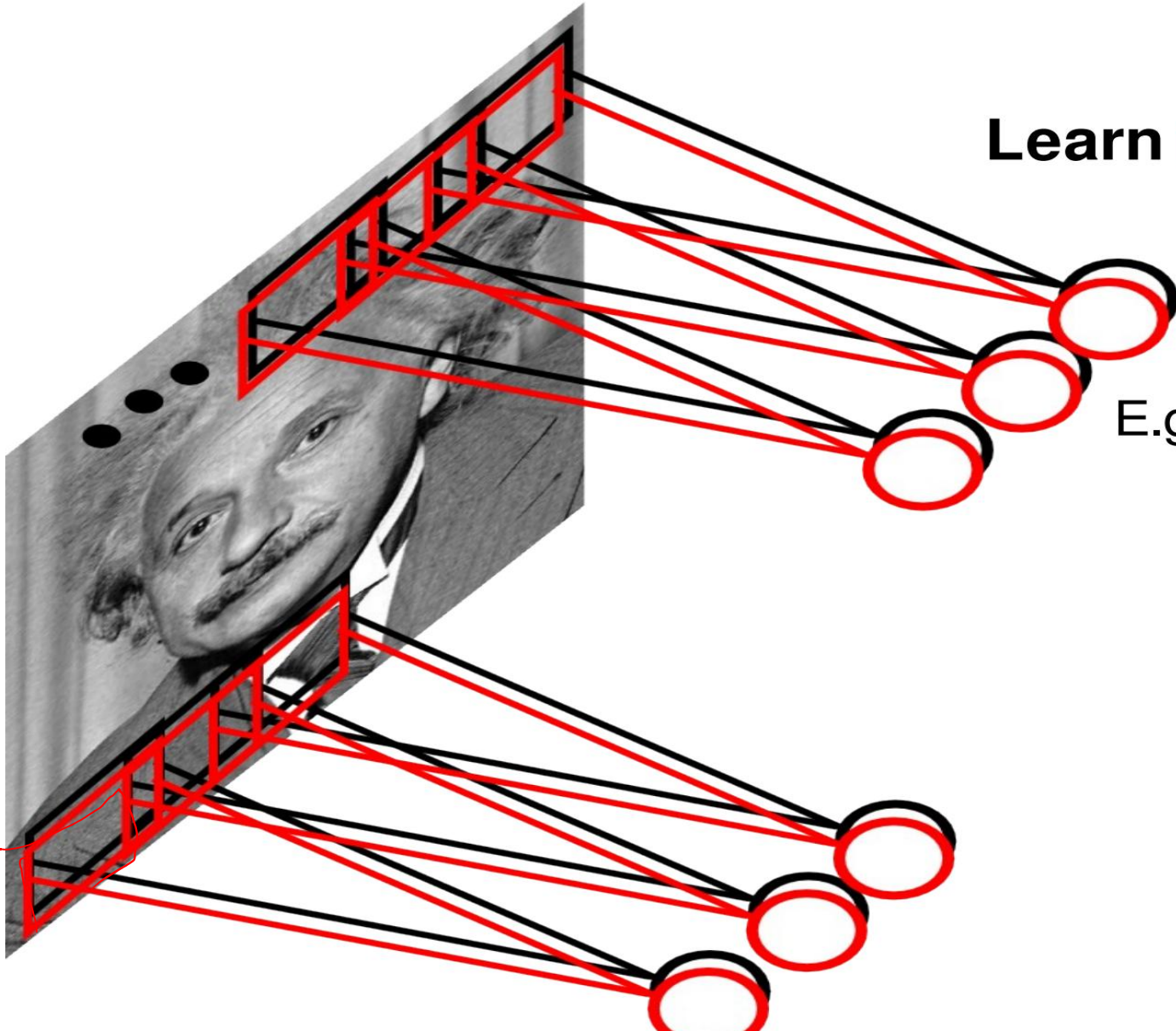




Convolutional Layer

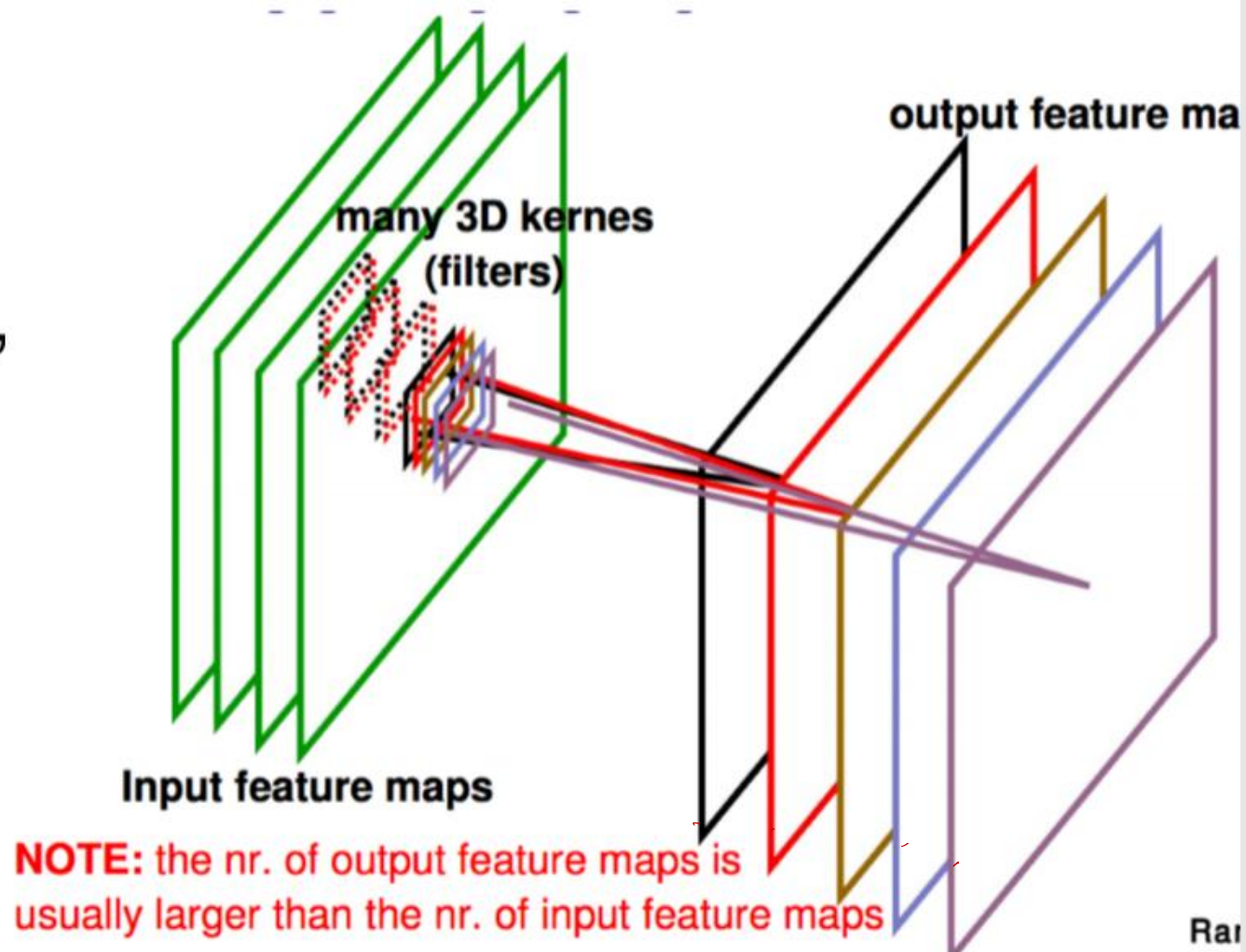
Learn **multiple filters**.

E.g.: 200x200 image
100 Filters
Filter size: 10x10
10K parameters



Using multiple filters

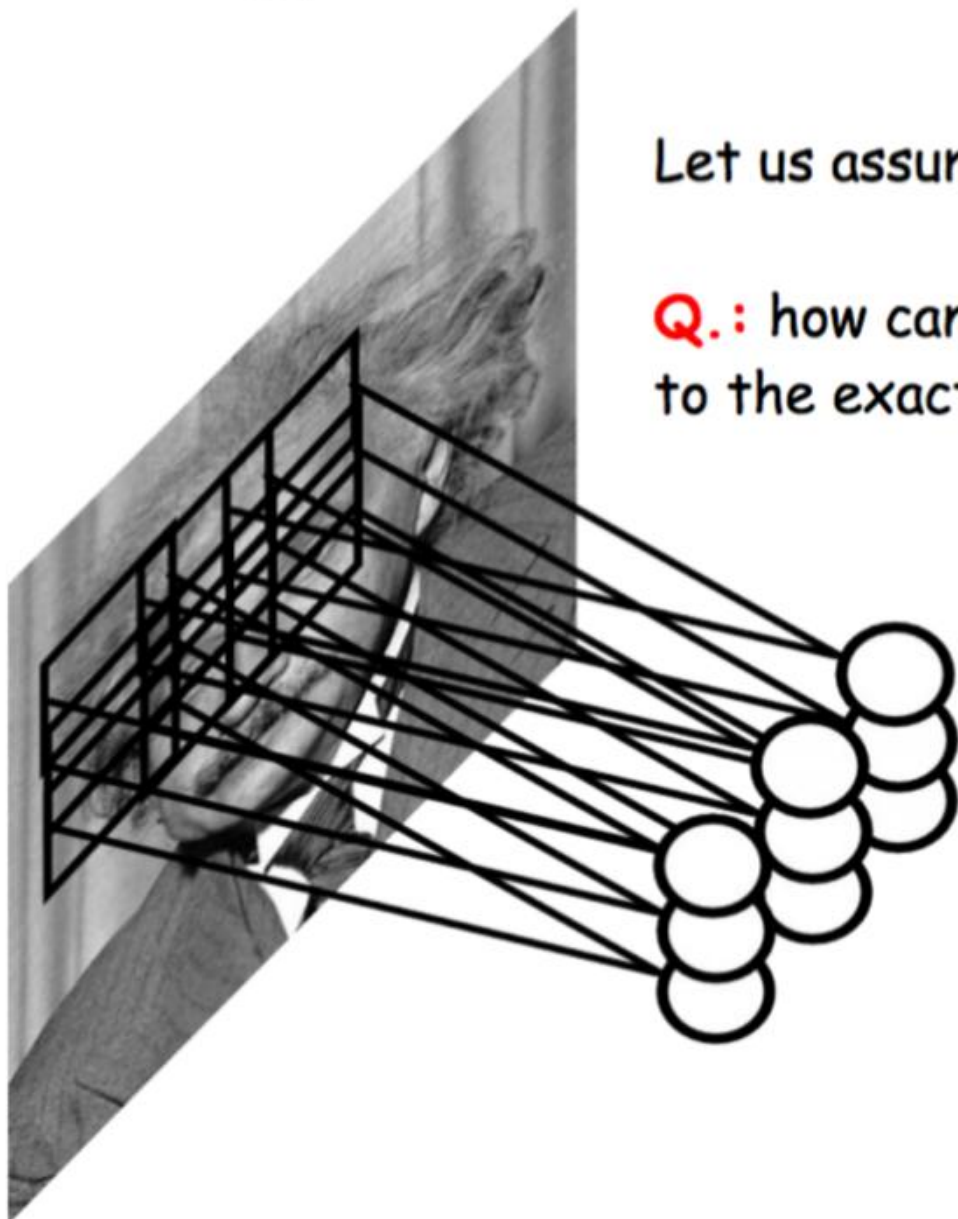
Each filter detects features in the output of previous layer.
So to capture different features, learn multiple filters.



Building Translation Invariance

Let us assume filter is an "eye" detector.

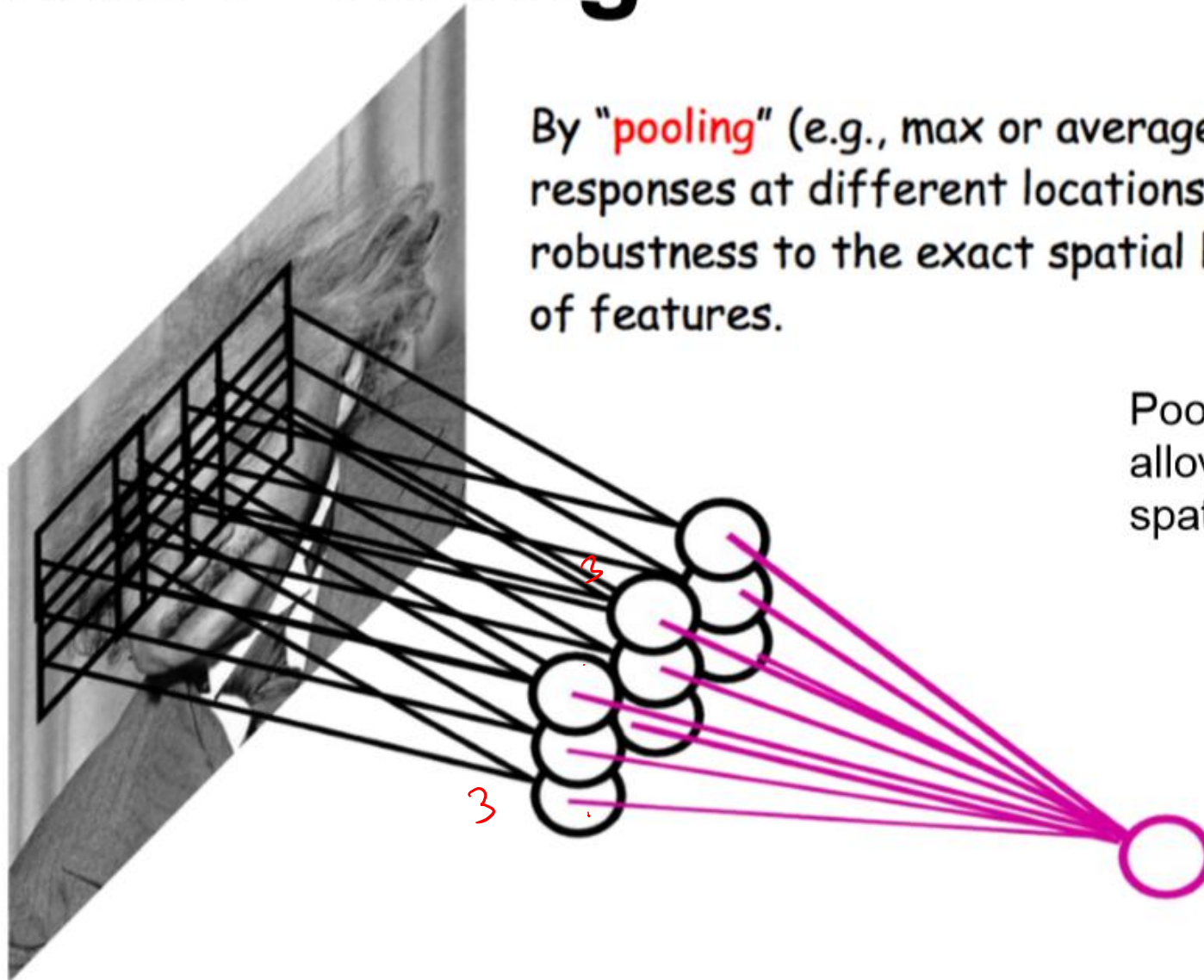
Q.: how can we make the detection robust to the exact location of the eye?

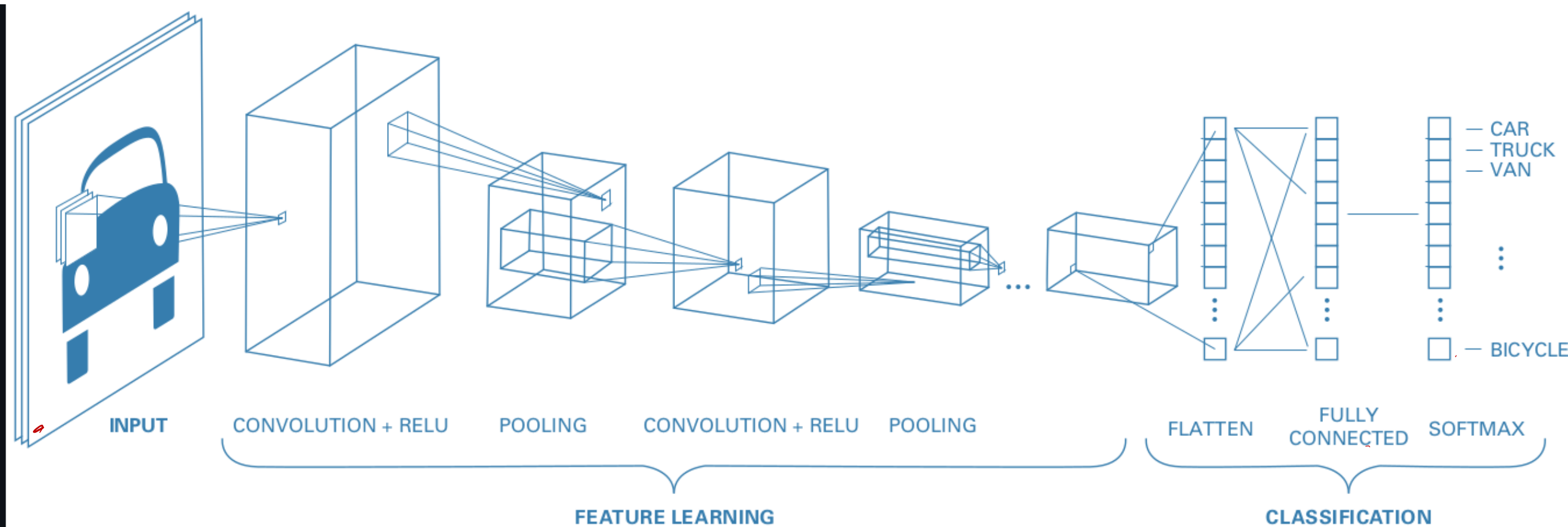


Building Translation Invariance via Spatial Pooling

By “pooling” (e.g., max or average) filter responses at different locations we gain robustness to the exact spatial location of features.

Pooling also subsamples the image, allowing the next layer to look at larger spatial regions.





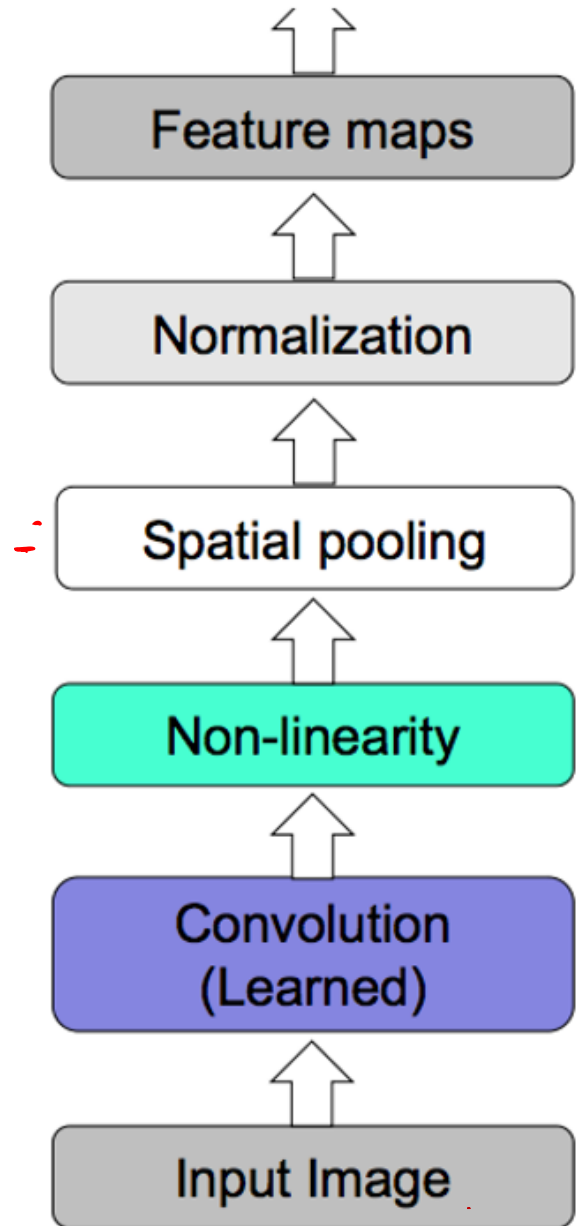
Summary of a typical convolutional layer

Doing all of this consists one layer.

- Pooling and normalization is optional.

Stack them up and train just like multi-layer neural nets.

Final layer is usually fully connected neural net with output size == number of classes



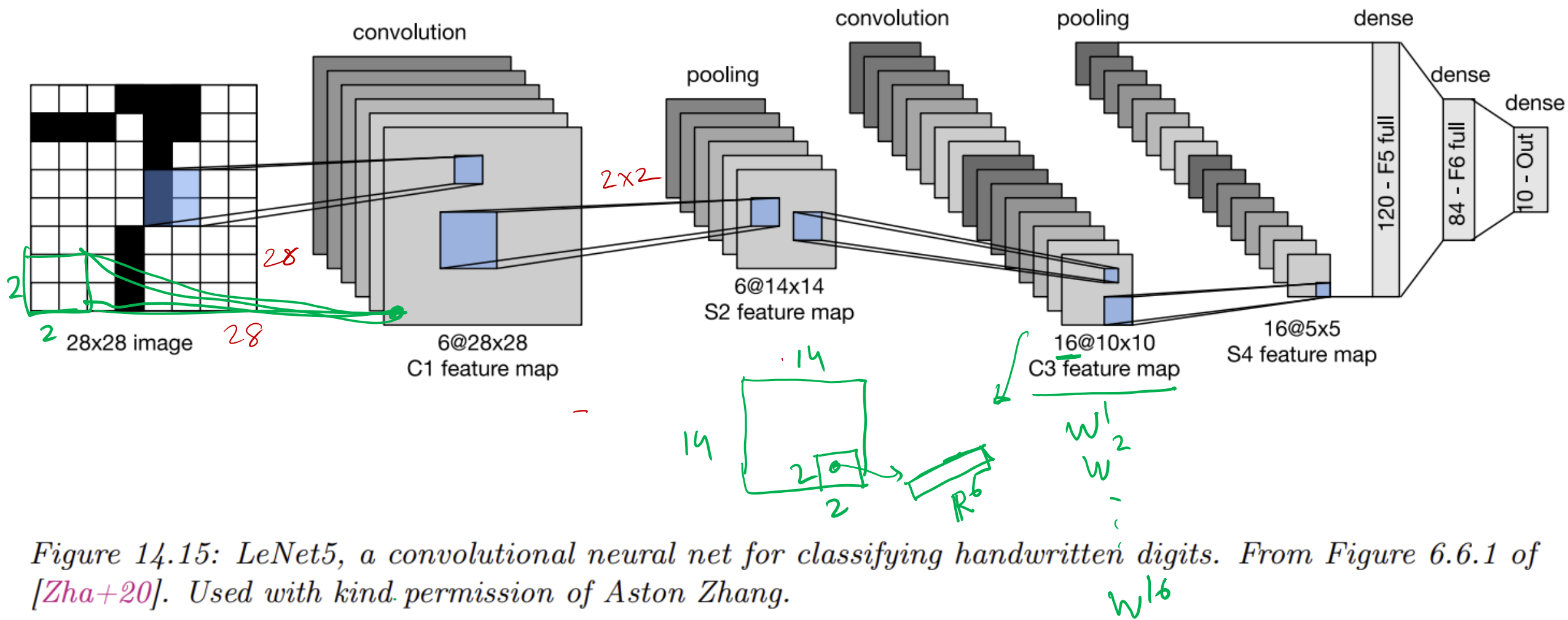


Figure 14.15: LeNet5, a convolutional neural net for classifying handwritten digits. From Figure 6.6.1 of [Zha+20]. Used with kind permission of Aston Zhang.

The Inception-Resnet V3 architecture

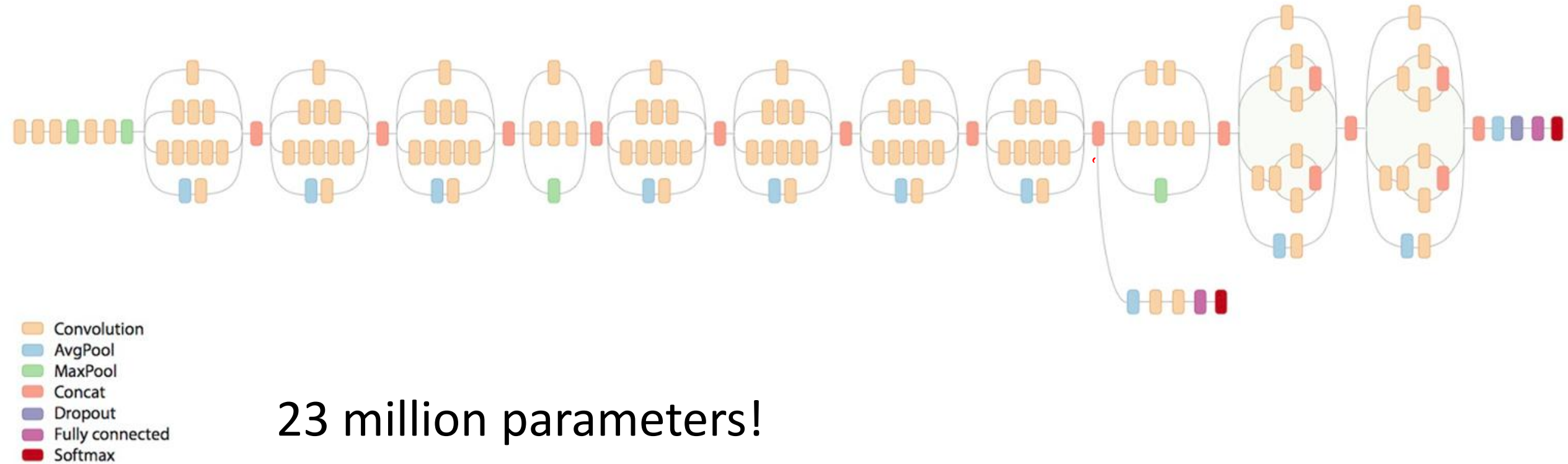


Image classification Code

- [Image classification | TensorFlow Core](#)

1st conv layer:

$$16 \times (3 \times 3 \times 3 + 1) = 448$$

bias.

2nd conv layer:

$$32 \times (3 \times 3 \times 16 + 1) = 4640$$

```
model = Sequential([
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
])
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
rescaling_1 (Rescaling)	(None, 180, 180, 3)	0
conv2d (Conv2D)	(None, 180, 180, 16)	448
max_pooling2d (MaxPooling2D)	(None, 90, 90, 16)	0
conv2d_1 (Conv2D)	(None, 90, 90, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 45, 45, 32)	0
conv2d_2 (Conv2D)	(None, 45, 45, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 22, 22, 64)	0
flatten (Flatten)	(None, 30976)	0
dense (Dense)	(None, 128)	3965056
dense_1 (Dense)	(None, 5)	645
=====		
Total params: 3,989,285		